

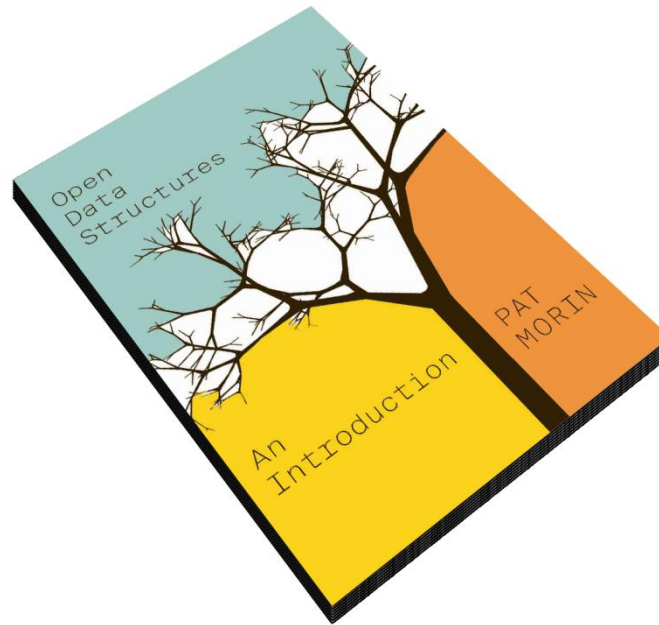
DualArrayDeque

COMP 2402

Abstract Data Types & Algorithms

Array-Based Lists

- Chapter 2



ArrayStack

We can't ignore resizing but worst case analysis is not very reflective of actual behaviour

Lemma 2.1: if an ArrayStack is created and any sequence of $m \geq 1$ calls to `add(i,x)/remove(i)` are performed, then the total time spent during the calls to `resize()` is $O(m)$

The **amortized cost** of resizing, taken over $m \geq 1$ calls to `add/remove`, is then $\frac{O(m)}{m} \in O(1)$

ArrayStack

Theorem 2.1: An ArrayStack implements the List interface. Ignoring the cost of `resize()`, an ArrayStack supports the operations:

- `get(i)` and `set(i,x)` in $O(1)$ time per operation,
- `add(i,x)` and `remove(i)` in $O(1 + n - i)$ time per operation.

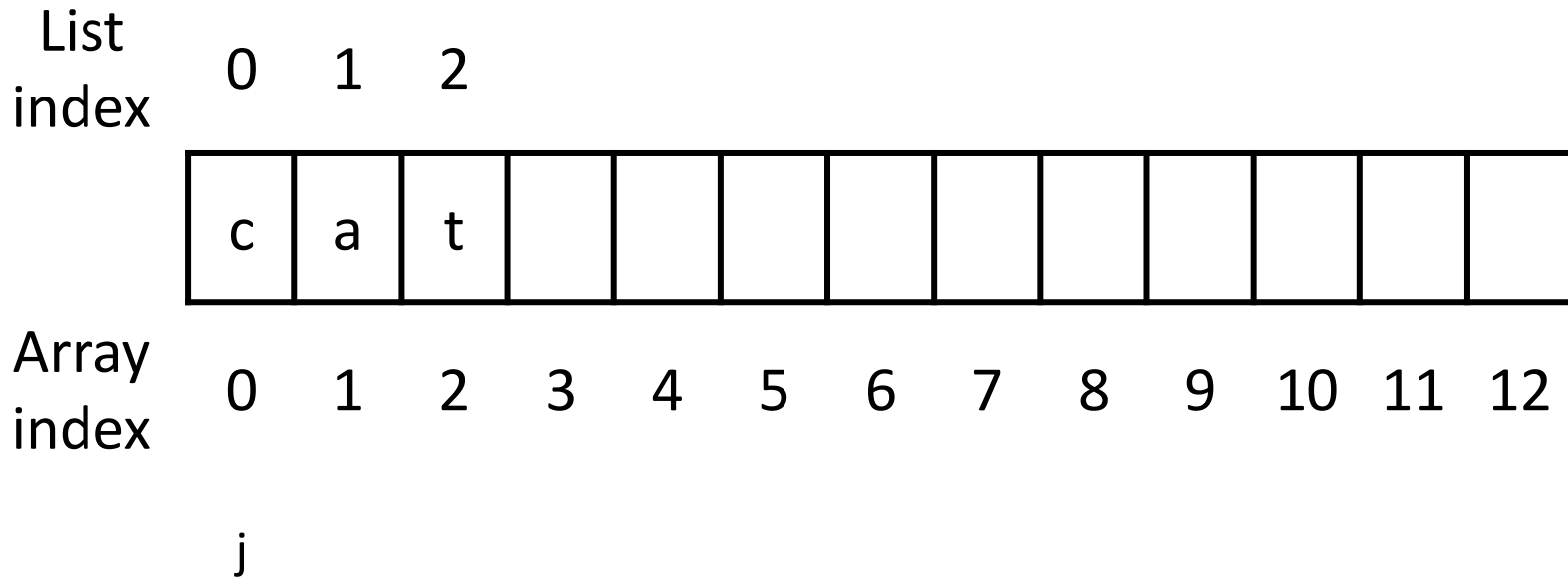
Furthermore, see Lemma 2.1... resizing has constant amortized cost.

FastArrayStack

ODS 2.2 FastArrayStack

`System.arraycopy`

ArrayQueue



```
add(0, 'c');  
add(1, 'a');  
add(2, 't');
```

```
add('c')  
add('a')  
add('t')
```

ArrayQueue

List
index

0 1 2

		t	o	e								
--	--	---	---	---	--	--	--	--	--	--	--	--

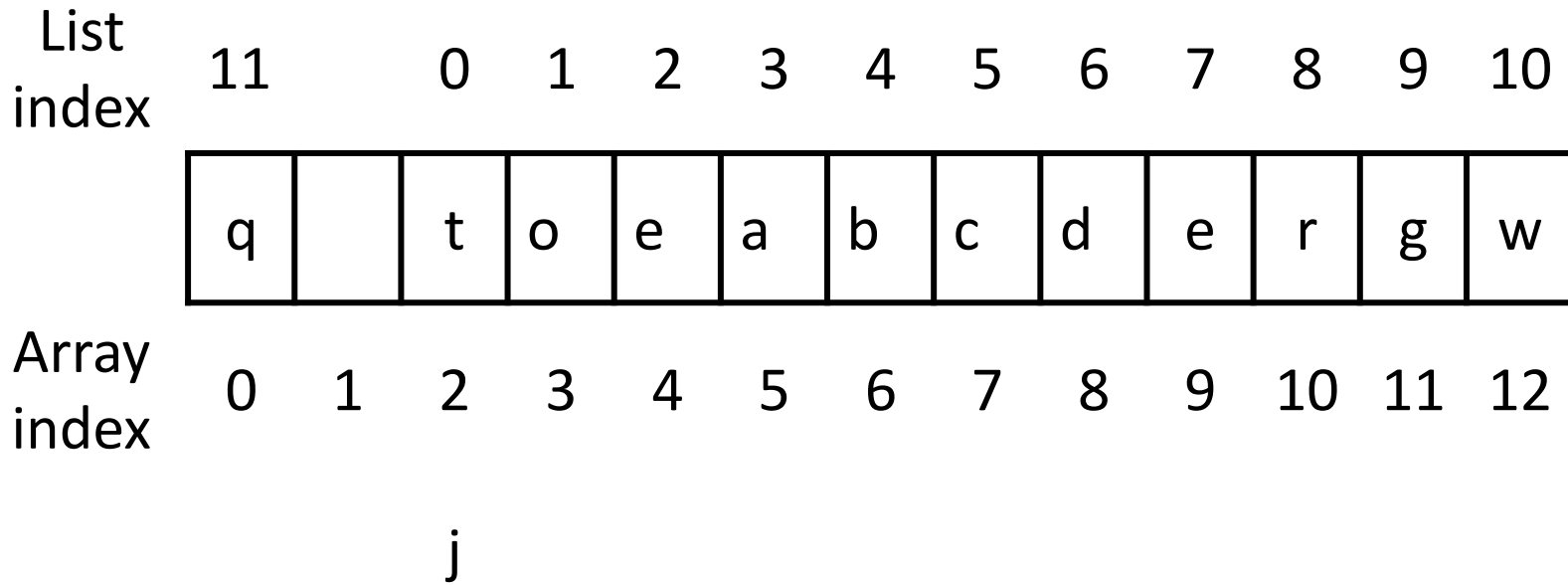
Array
index

0 1 2 3 4 5 6 7 8 9 10 11 12

j

```
remove(0);  
add(2, 'o');  
remove(0);  
add(2, 'e');
```

ArrayQueue



enqueue a,b,c,d,e,r,g,w,q

ArrayQueue

Theorem 2.2: An ArrayQueue implements (FIFO) Queue interface. Ignoring the cost of `resize()`, an ArrayQueue supports the operations `add(x)` and `remove()` in $O(1)$ time per operation.

In addition, starting with an empty ArrayQueue, any sequence of $m \geq 1$ add/remove operations results in $O(m)$ time spent on calls to `resize()`

ArrayDeque

ODS 2.4 ArrayDeque

Needs to have fast Deque operations

ArrayDeque

List
index

0 1 2 3 4 5 6

		t	r	o	u	b	l	E				
--	--	---	---	---	---	---	---	---	--	--	--	--

Array
index

0 1 2 3 4 5 6 7 8 9 10 11 12

j

$(j+n-1) \% 13$

`add(0, 'X');` `(addFront)`

ArrayDeque

List
index

0 1 2 3 4 5 6 7

	X	t	r	o	u	b	l	E				
--	---	---	---	---	---	---	---	---	--	--	--	--

Array
index

0 1 2 3 4 5 6 7 8 9 10 11 12

j

$(j+n-1) \% 13$

`add(0, 'X');` `(addFront)`

ArrayDeque

List
index

0 1 2 3 4 5 6 7

	t	X	r	o	u	b	l	E				
--	---	---	---	---	---	---	---	---	--	--	--	--

Array
index

0 1 2 3 4 5 6 7 8 9 10 11 12

j

$(j+n-1) \% 13$

`add(8, 'Q');` `(addBack)`

ArrayDeque

List
index

0 1 2 3 4 5 6 7 8

	t	X	r	o	u	b	l	E	Q			
--	---	---	---	---	---	---	---	---	---	--	--	--

Array
index

0 1 2 3 4 5 6 7 8 9 10 11 12

j

$(j+n-1) \% 13$

`add(8, 'Q');` `(addBack)`

ArrayDeque

List
index

0 1 2 3 4 5 6 7 8

	t	X	r	o	u	b	l	E	Q			
--	---	---	---	---	---	---	---	---	---	--	--	--

Array
index

0 1 2 3 4 5 6 7 8 9 10 11 12

j

$(j+n-1) \% 13$

`add(2, 'Z');` treat it like a List

ArrayDeque

List index	0	1	2	3	4	5	6	7	8	9			
	t	X	Z	r	o	u	b	l	E	Q			
Array index	0	1	2	3	4	5	6	7	8	9	10	11	12
	j								$(j+n-1) \% 13$				

`add(2, 'Z');` treat it like a List

ArrayDeque

List index	0	1	2	3	4	5	6	7	8	9			
	t	X	Z	r	o	u	b	l	E	Q			
Array index	0	1	2	3	4	5	6	7	8	9	10	11	12
	j						$(j+n-1) \% 13$						

`add(6, 'Y');` treat it like a List

ArrayDeque

List index	0	1	2	3	4	5	6	7	8	9	10		
	t	X	Z	r	o	u	Y	b	l	E	Q		
Array index	0	1	2	3	4	5	6	7	8	9	10	11	12
	j						$(j+n-1) \% 13$						

`add(6, 'Y');` treat it like a List

ArrayDeque

- Remove from first half of array
 - This is same as ArrayQueue
- Remove from second half of array
 - This is same as ArrayStack but using $\% a.length$
- Add in first half of array
 - Special case to consider (because $\% \neq mod$)
- Add in second half of array
 - This is same as ArrayQueue

ArrayDeque

Theorem 2.3: An ArrayDeque implements the List interface. Ignoring the cost of `resize()`, an ArrayDeque supports the operations

- `get(i)/set(i,x)` in $O(1)$ time per operation, and
- `add(i,x)/remove(i)` in $O(1 + \min\{i, n - i\})$ time per operation

In addition, starting with an empty ArrayDeque, any sequence of $m \geq 1$ add/remove operations results in $O(m)$ time spent on calls to `resize()`

DualArrayDeque

ODS 2.5 DualArrayDeque

- Implements a Deque with TWO ArrayStacks
- Nice example of using known data structures as building blocks for other data structures!

DualArrayDeque

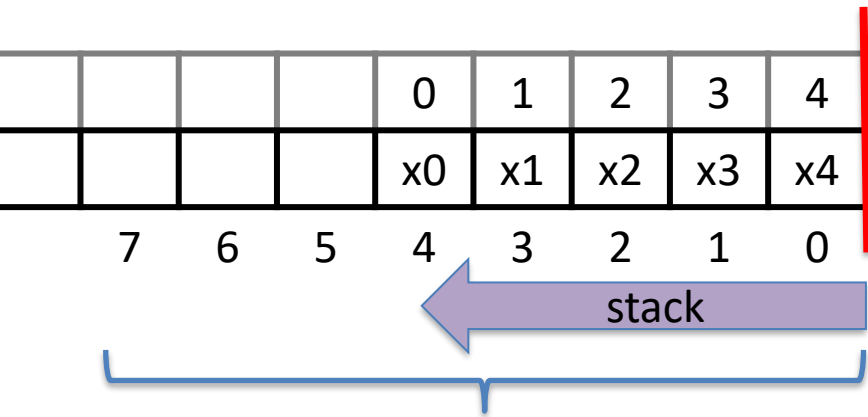
It is often useful to use known data structures as building blocks to create larger and more complex data structures

A Deque is a List that needs fast access (adding/removing) to **both** the front and back of the list

Let's split the list into two halves (front/back)

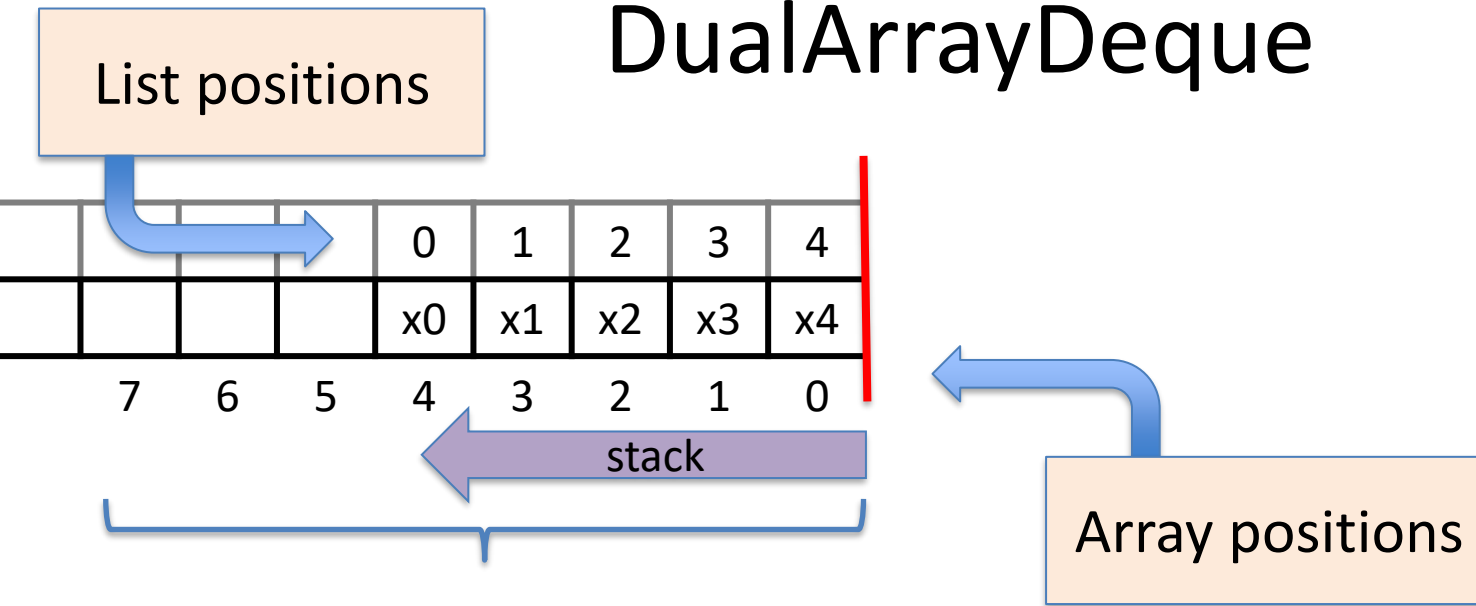
$$X_0, X_1, X_2, \dots, X_{n/2-1}, X_{n/2}, X_{n/2+1}, \dots, X_{n-1}$$

DualArrayDeque



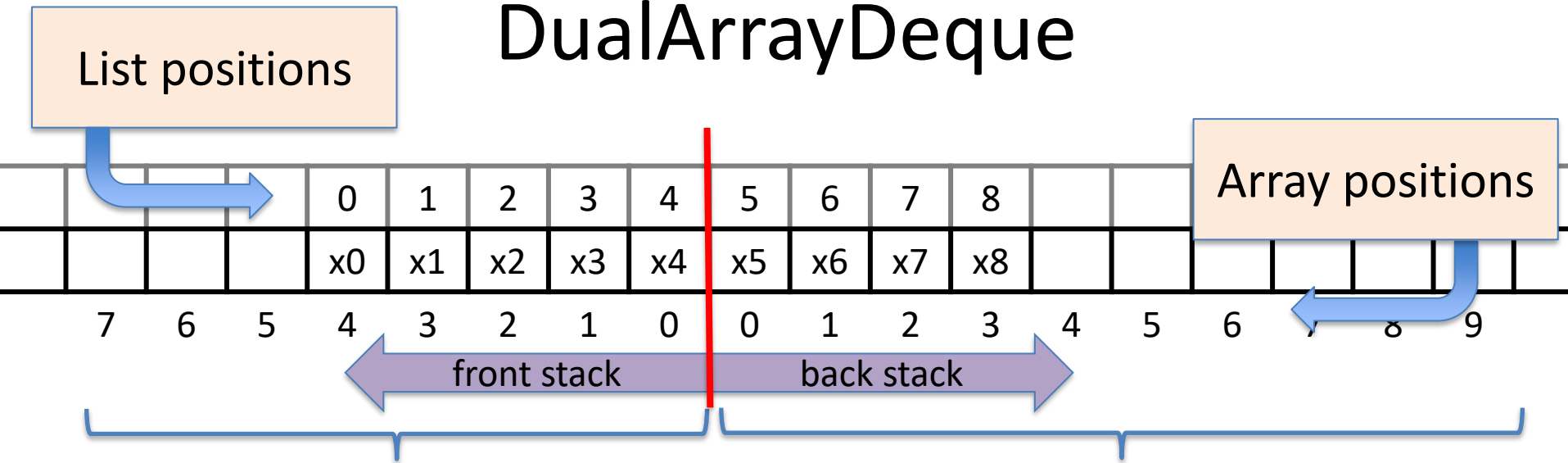
ArrayStack for front of list
(in reverse order)

DualArrayDeque



ArrayStack for front of list
(in reverse order)

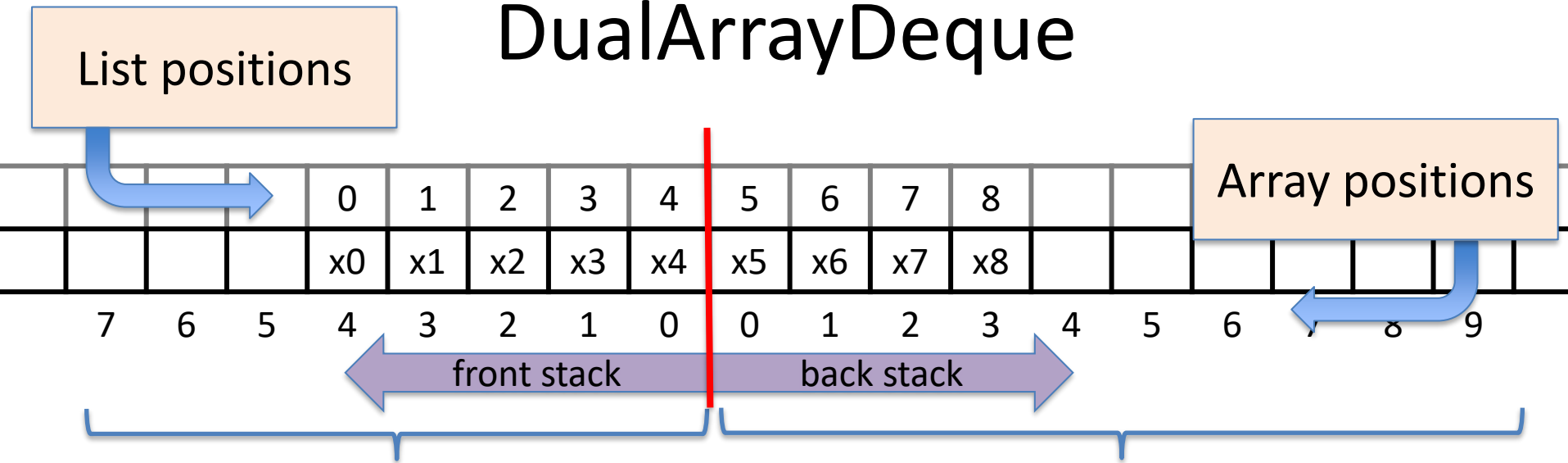
DualArrayDeque



ArrayStack for front of list
(in reverse order)

ArrayStack for back of list

DualArrayDeque



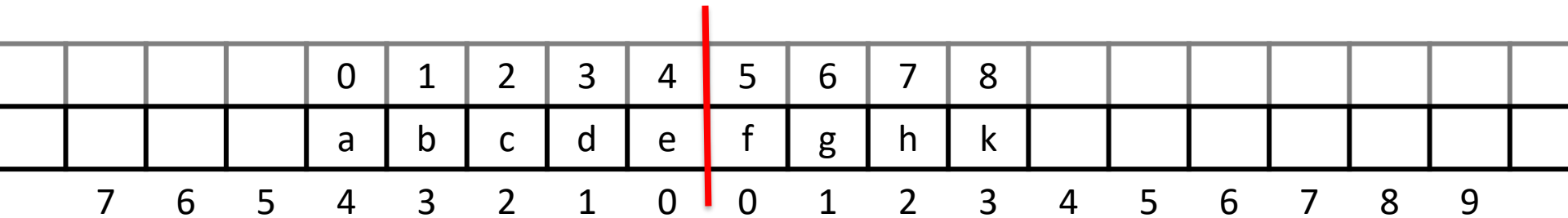
ArrayStack for front of list
(in reverse order)

ArrayStack for back of list

front
 $f = \text{front.size}()$

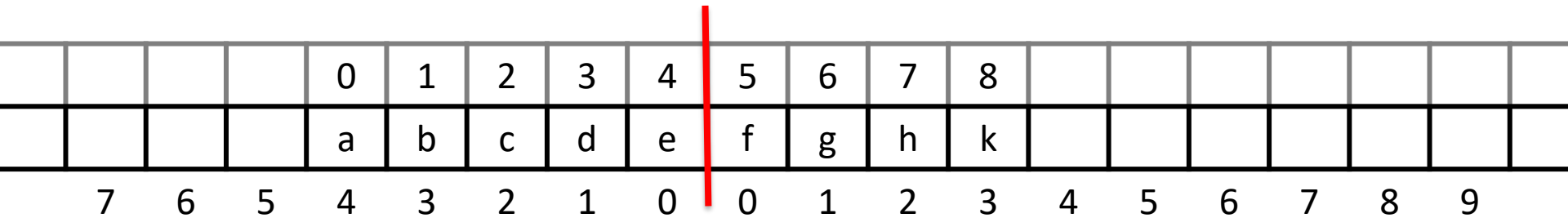
back
 $b = \text{back.size}()$

DualArrayDeque

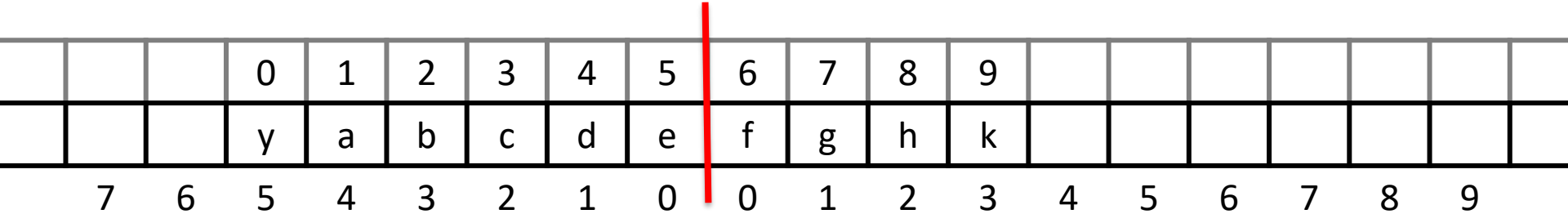


Add y to front of list `add(0,y)`

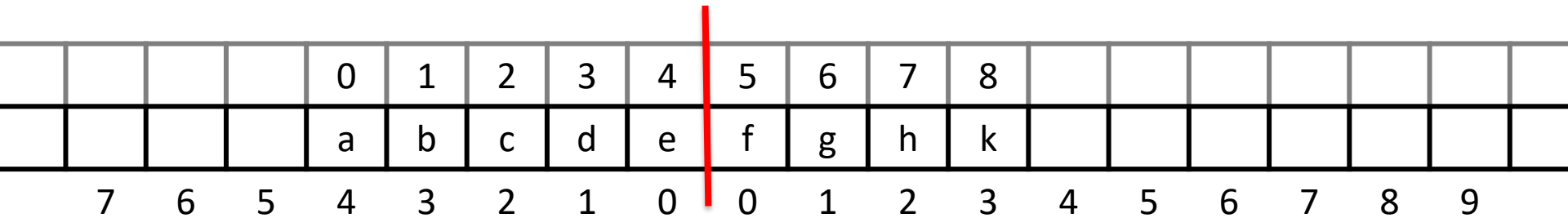
DualArrayDeque



Add y to front of list add(0,y)

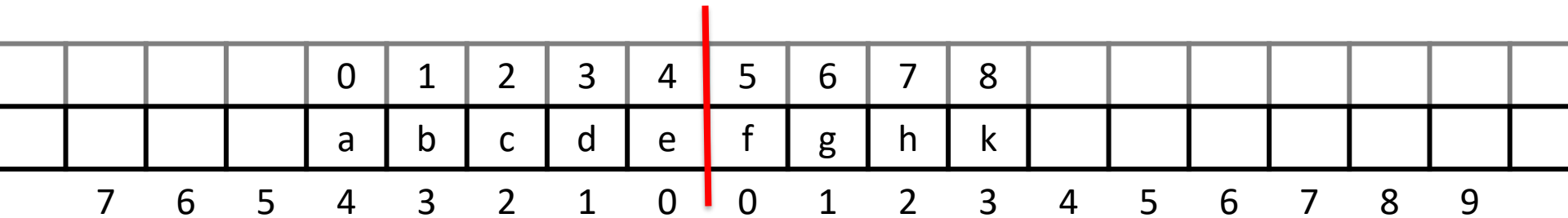


DualArrayDeque

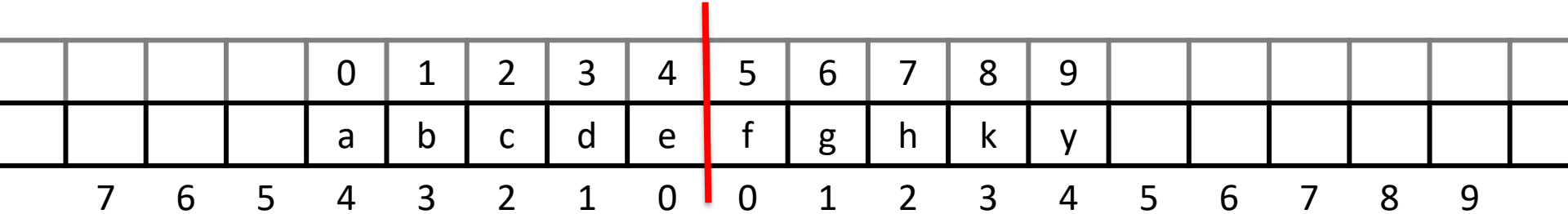


Add y to back of list `add(n,y)`

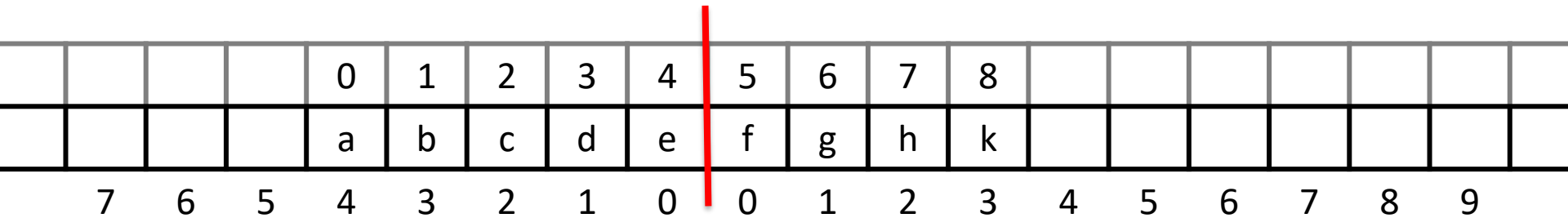
DualArrayDeque



Add y to back of list add(n,y)

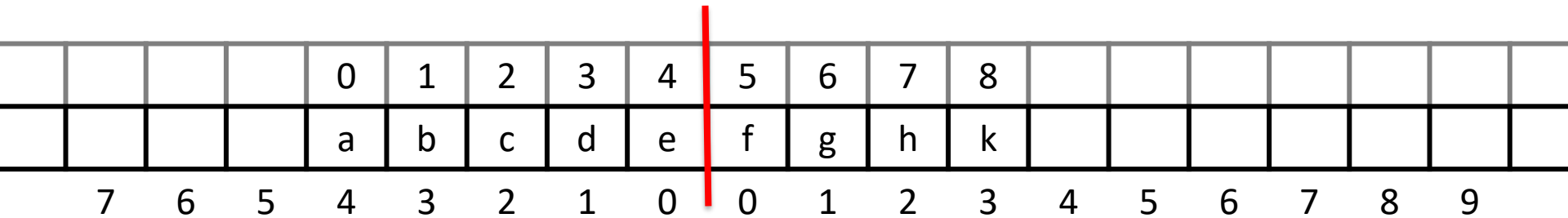


DualArrayDeque

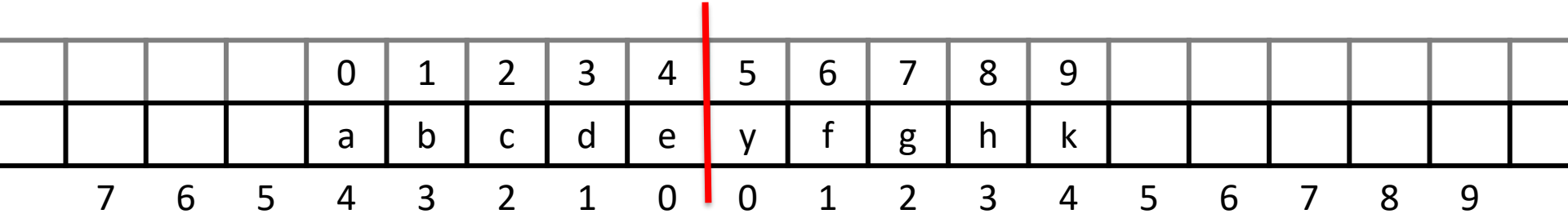


Add y to position 5 of the list `add(5,y)`

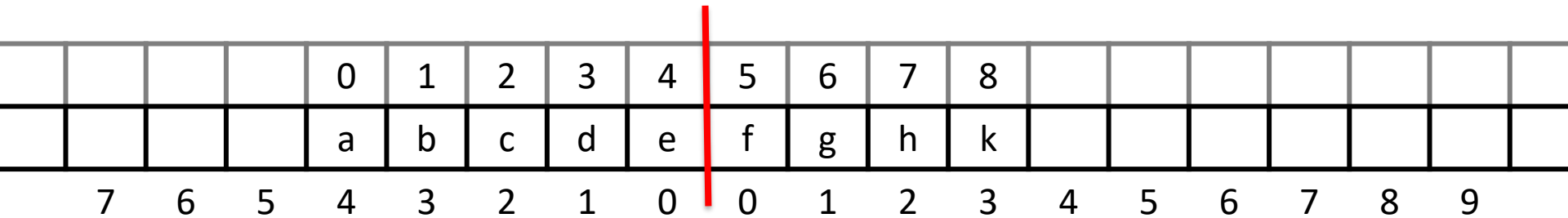
DualArrayDeque



Add y to position 5 of the list `add(5,y)`



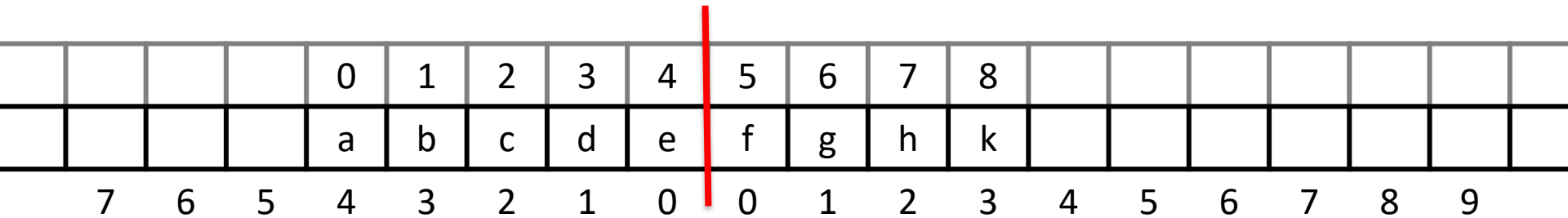
DualArrayDeque



We use two ArrayStacks to implement a list

What is a potential problem here?

DualArrayDeque



We'll keep the two ArrayStacks “balanced”

$$3f \geq b$$

$$3b \geq f$$

Rebalance the stacks when violated

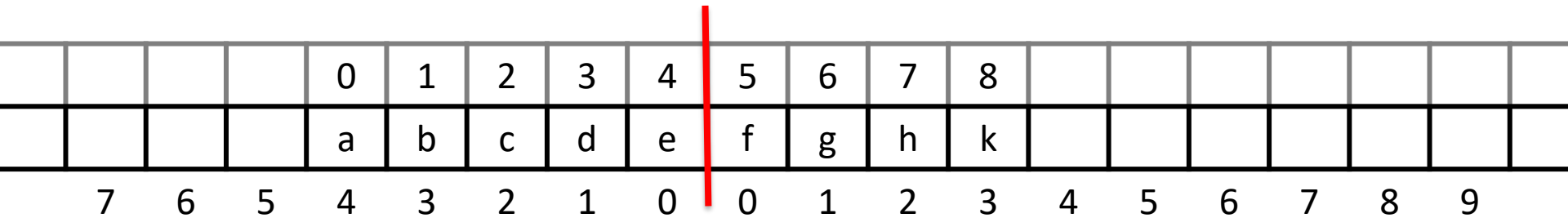
DualArrayDeque

				0	1	2	3	4		5	6	7	8						
				a	b	c	d	e		f	g	h	k						
7	6	5	4	3	2	1	0		0	1	2	3	4	5	6	7	8	9	

Exercise: Prove that

1. if $0 \leq i < \frac{n}{4}$ then we are in the front stack
2. if $\frac{3}{4}n < i \leq n$ then we are in the back stack

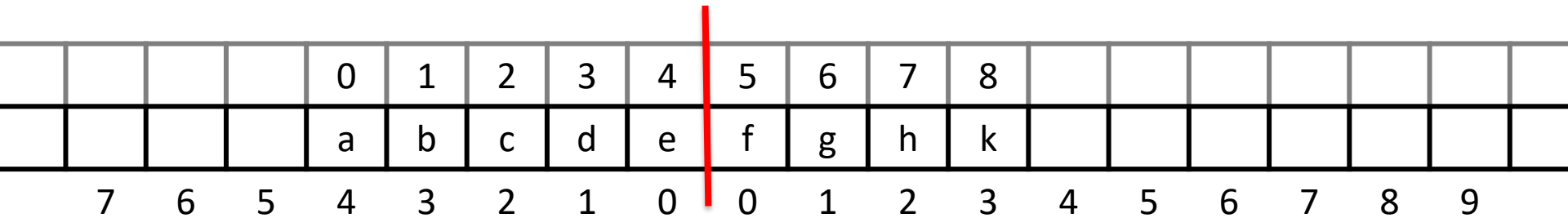
DualArrayDeque



What is cost of add/remove when $i < \frac{1}{4}n$

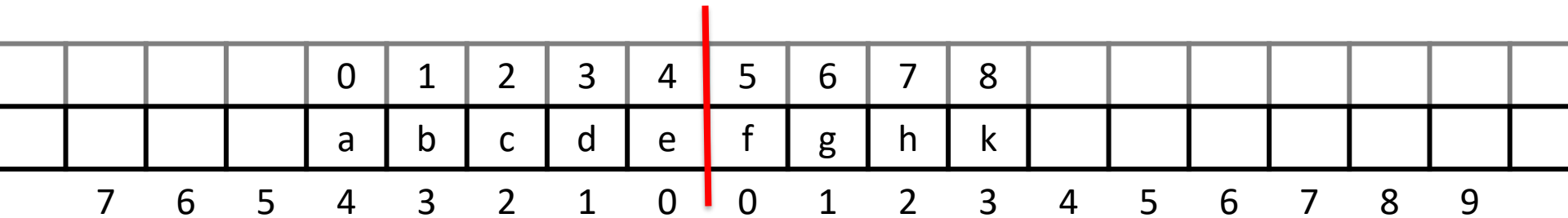
-

DualArrayDeque



What is cost of add/remove when $i > \frac{3}{4}n$

DualArrayDeque



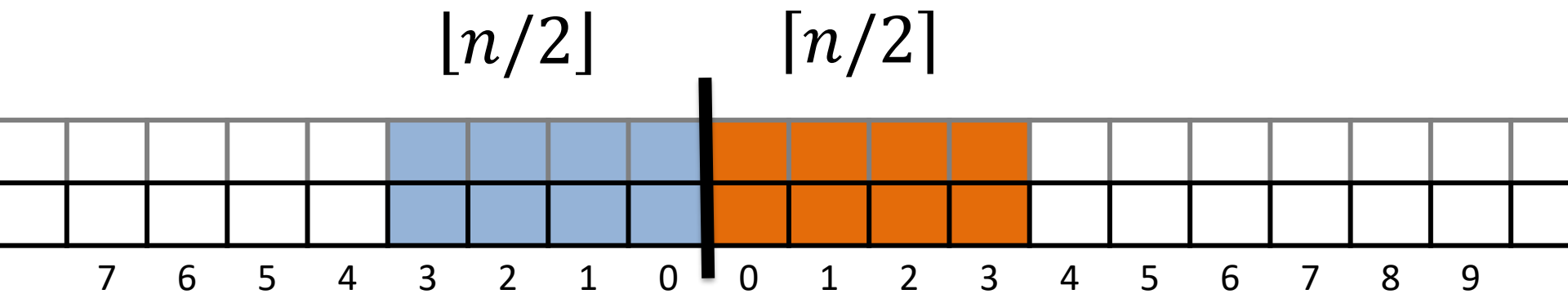
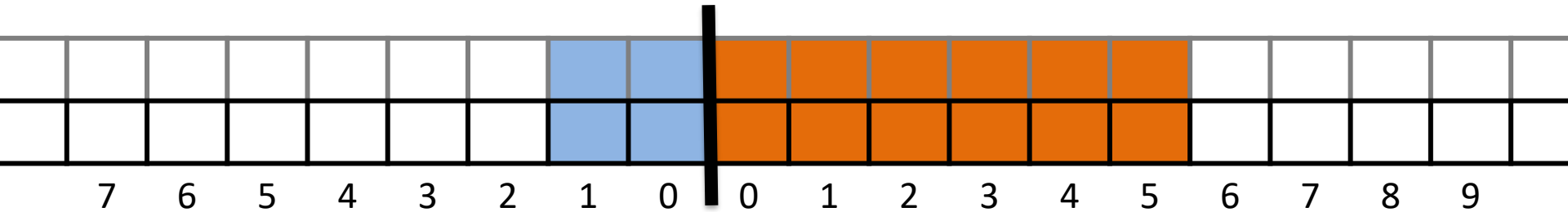
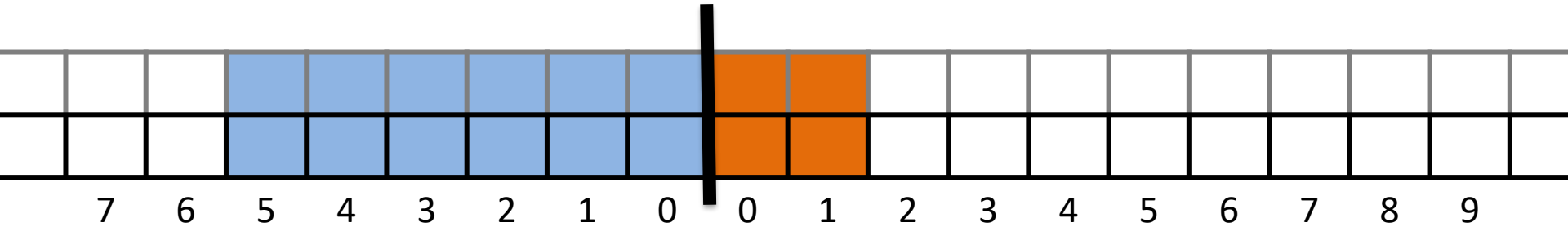
Consider $\frac{1}{4}n \leq i \leq \frac{3}{4}n$

DualArrayQueue

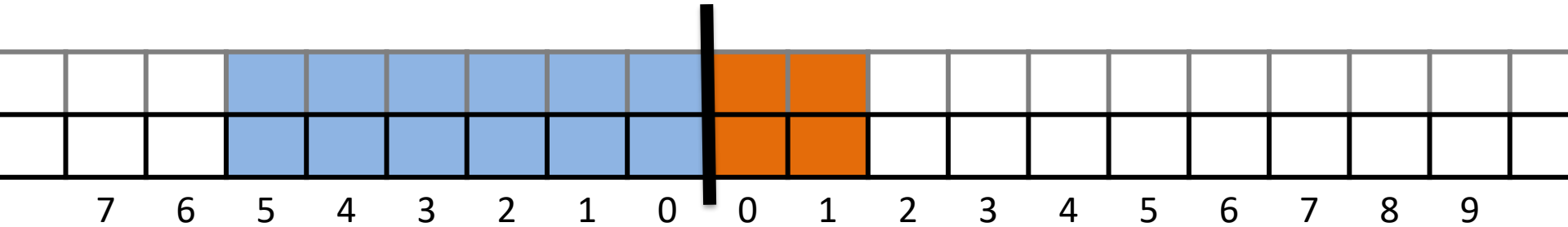
Theorem 2.4: A DualArrayQueue implements the List interface. Ignoring the cost of `resize()` and `balance()`, a DualArrayQueue supports the operations

- `get(i)/set(i,x)` in $O(1)$ time per operation, and
- `add(i,x)/remove(i)` in $O(1 + \min\{i, n - i\})$ time per operation

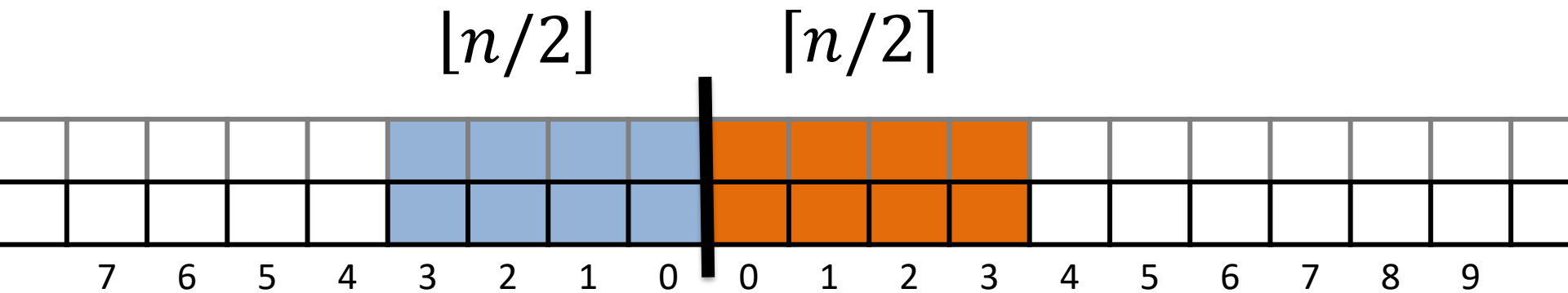
DualArrayDeque



DualArrayDeque



What is cost of balancing?



Potential Method

Define a **potential function** for the data structure to be the absolute difference of the sizes of the two stacks

$$\Phi = |f - b| \geq 0$$

Any add/remove operation can only increase the potential by at most 1. (Other operations do not affect Φ)

Minimum number of add/remove calls between two states is at least $|\Phi(s_1) - \Phi(s_0)|$

Potential Method

The potential just after a balancing is

$$\Phi_0 = ||n_0/2] - [n_0/2|| \leq 1$$

The potential just before the next balancing is (assume wlog that balancing is triggered by $3b < f$)

$$n = f + b < f + \frac{1}{3}f = \frac{4}{3}f \quad (\text{so } f > \frac{3}{4}n)$$

$$\Phi_1 = f - b > f - \frac{1}{3}f = \frac{2}{3}f > \frac{2}{3} \cdot \frac{3}{4}n = \frac{1}{2}n$$

Potential Method

The potential difference between last balancing and current balancing is then

$$\Delta = |\Phi_1 - \Phi_0| > \frac{1}{2}n - 1$$

So we see that least $n/2 - 1$ add/remove operations were needed since the last balancing.

Total time spent on balance, over m add/removes, is then $O(m)$, just as with the ArrayStack.

DualArrayDeque

Theorem 2.4: A DualArrayDeque implements the List interface. Ignoring the cost of `resize()` and `balance()`, a DualArrayDeque supports the operations

- `get(i)/set(i,x)` in $O(1)$ time per operation, and
- `add(i,x)/remove(i)` in $O(1 + \min\{i, n - i\})$ time per operation

In addition, starting with an empty DualArrayDeque, any sequence of $m \geq 1$ add/remove operations results in $O(m)$ time spent on calls to `resize()` and `balance()`

Wasted Space?

How much space is wasted (at any time) in these data structures?

- ArrayStack
- ArrayQueue
- ArrayDeque
- DualArrayDeque